

Real-Time Traffic Anomaly Detection with MLOps

Abstract

An end-to-end MLOps pipeline that detects traffic anomalies from Estonian census data by processing hourly measurements of volume, speed, and vehicle types, enabling proactive traffic management through automated model deployment and monitoring.

Workflow

The system continuously ingests new data files, extracts temporal and traffic features, applies an anomaly detection model, exposes predictions through an API, and monitors performance to trigger retraining with fresh data and operator feedback.

Purpose	Technology
Baseline training	Scikit-learn
Experiment tracking	MLflow
Scheduling	Prefect (hourly job)
Anomaly scoring	FastAPI microservice
Drift detection	Evidently AI
Dashboard	Streamlit + Folium (map) (maybe)
Storage	Parquet
Deployment	Docker
Version control	Git, DVC, MLflow

Data: <https://andmed.eesti.ee/datasets/liiklusloenduse-andmed>

Model

The system uses a **multivariate anomaly detection** approach to identify unusual traffic patterns in volume, speed, and vehicle composition. The pipeline is designed for incremental learning, allowing new data (2025+) to be added via retraining while preserving 2026 for testing.

Training Strategy

- **Training Data:** 2018–2024 (all available historical data)
- **Validation Data:** 2025 (used for model tuning and drift detection)
- **Test Data:** 2026 (Jan–Apr, reserved for final evaluation)
- **Retraining:** Hourly jobs (Prefect) add new 2025+ data and retrain the model

Data Handling

- **Missing Speed Data:** For stations with missing speed columns (25d51, 0e481, ff2e7, fde61):
 - **Option 1:** Exclude speed features and train on vehicle counts only (1–10)
 - **Option 2:** Impute missing speed values using historical medians per `id` + `hour` from 2018–2023
 - **Option 3:** Train separate models for stations with/without speed data

Feature Engineering

```
# Core features derived from raw columns
features = {
    "total_vehicles": sum(vehicle_cols_1_to_10),          # Sum of columns
    1-10
    "avg_speed_bin": weighted_avg(speed_cols),          # Approximate
    average speed
    "pct_heavy_vehicles": (cols_6_7_8) / total_vehicles, # Truck ratio
    "hour": extract_hour(aeg),                          # Hour of day
    "day_of_week": extract_dow(aeg),                    # Day of week
    "is_weekend": day_of_week in [5, 6],                # Weekend flag
    "is_holiday": check_estonian_holidays(aeg),         # Holiday flag
    "rolling_avg_24h": mean(total_vehicles, window=24h), # 24h moving
    average
    "lane_ratio": kanal_volume / id_total_volume        # Per-lane
    imbalance
}
```

Model Architecture

Algorithm

The system uses **Isolation Forest** from Scikit-learn, an unsupervised anomaly detection algorithm that works by:

1. Randomly selecting features and split values to isolate observations
2. Building an ensemble of isolation trees (100 trees by default)
3. Computing anomaly scores based on path lengths - shorter paths indicate anomalies

Why Isolation Forest?

- **Unsupervised**: No need for labeled anomaly data
- **Multivariate**: Handles all 9 features simultaneously
- **Efficient**: Linear time complexity $O(n)$ for training
- **Interpretable**: Anomaly scores indicate degree of anomaly
- **Scalable**: Works well with large datasets (11M+ rows)

Model Configuration

Parameter	Value	Description
<code>`n_estimators`</code>	100	Number of trees in the forest
<code>`contamination`</code>	0.01	Expected % of anomalies (1%)
<code>`max_samples`</code>	auto	Samples per tree (min(256, n_samples))
<code>`random_state`</code>	42	Reproducibility seed
<code>`n_jobs`</code>	-1	Use all CPU cores

Training Data

- **Input**: 11,205,232 rows (2018-2024)

```
- **Features**: 9 engineered features
- **Training Time**: ~30-60 seconds
- **Model Size**: ~1.5 MB

## Model Outputs

### Prediction Response
The API returns a JSON response with the following fields:

```json
{
 "anomaly_score": -0.4308,
 "prediction": 1,
 "is_anomaly": false,
 "anomaly_score_normalized": 0.1384
}
```

## Output Fields

Field	Type	Range	Description
<code>anomaly_score</code>	float	$(-\infty, 0.5]$	Raw anomaly score from Isolation Forest. <b>Lower values = more anomalous</b>
<code>prediction</code>	int	$\{-1, 1\}$	Binary prediction: -1 = anomaly, 1 = normal
<code>is_anomaly</code>	bool	$\{\text{true}, \text{false}\}$	Human-readable anomaly flag
<code>anomaly_score_normalized</code>	float	$[0, 1]$	Normalized score where 0 = most anomalous, 1 = most normal

## Score Interpretation

Score Range	Interpretation
$\text{anomaly\_score} < -0.5$	<b>High confidence anomaly</b>
$-0.5 \leq \text{anomaly\_score} < 0$	<b>Moderate anomaly</b>
$0 \leq \text{anomaly\_score} \leq 0.5$	<b>Normal</b>

The anomaly score represents the **path length** through the isolation trees:

- **Shorter paths** → Fewer splits needed to isolate → More anomalous → **Lower score**
- **Longer paths** → More splits needed → More normal → **Higher score (closer to 0.5)**

## Normalized Score

The `anomaly_score_normalized` maps the raw score to a 0-1 range:

- **0.0**: Most anomalous observation in the training set
- **1.0**: Most normal observation in the training set

This makes it easier to set thresholds (e.g., alert when score < 0.2).

### Thresholds

The model uses a default contamination of 1%, meaning approximately 1% of observations are flagged as anomalies. You can adjust this by:

- 1. Changing the `contamination` parameter during training
- 2. Applying a custom threshold on `anomaly_score_normalized` in your application

### Feature Importance

The Isolation Forest doesn't provide direct feature importance, but based on data analysis:

Feature	Expected Impact	Notes
<code>total_vehicles</code>	High	Sudden traffic volume changes are strong anomaly indicators
<code>avg_speed</code>	High	Unusual speed patterns (too fast/slow) flag anomalies
<code>hour</code>	Medium	Time-of-day patterns are learned from historical data
<code>day_of_week</code>	Medium	Weekend vs weekday traffic differences
<code>pct_heavy_vehicles</code>	Medium	Unexpected heavy vehicle ratios
<code>is_weekend</code>	Low	Captured by <code>day_of_week</code>
<code>is_holiday</code>	Low	Learned from historical holiday patterns
<code>rolling_avg_24h</code>	High	Deviations from 24-hour trends are anomalous
<code>lane_ratio</code>	Medium	Unusual lane distribution patterns

### Example Anomalies

#### Scenario 1: Sudden Traffic Drop

```
{
 "total_vehicles": 0,
 "avg_speed": 0,
 "hour": 10,
 "rolling_avg_24h": 500
}
```

**Result:** High anomaly score (likely accident or sensor failure)

#### Scenario 2: Unexpected Congestion

```
{
 "total_vehicles": 1000,
 "avg_speed": 15,
 "hour": 3,
 "rolling_avg_24h": 200
}
```

**Result:** High anomaly score (unusual for 3 AM)

Scenario 3: Speed Sensor Failure

```
{
 "total_vehicles": 500,
 "avg_speed": 0,
 "hour": 14
}
```

**Result:** Medium anomaly score (speed=0 with traffic present)

Team: Albert Wihler, Tanel Pastarus

Data Analysis Findings

Dataset Overview

- **Source:** Estonian Transport Administration traffic census data (Liiklusloenduse andmed)
- **Period:** 2018-2026 (hourly measurements)
- **Total Volume:** 14,076,486 rows across 9 yearly CSV files
- **Structure:** Each row = 1 counter device × 1 lane × 1 hour

Column Structure (24 columns per file)

Category	Columns	Description
Identifiers	id, kanal, aeg	Device ID, lane, timestamp
Vehicle Types	1-10	Motorcycle, Car, Heavy Van, Rigid, Articulated HGV, Minibus, Bus/Coach
Speed Ranges	<40Kph to =>130	11 speed bins (10 km/h increments)

Data Quality Issues

1. Missing Speed Data (2024-2026)

**Root Cause:** Sensor failure, not processing error.

Year	Missing Rows	Affected IDs	Impact
2024	2,016	25d51	Speed cols missing, vehicle counts intact
2025	19,740	25d51, 0e481, ff2e7	Speed cols missing, vehicle counts intact
2026	5,182	25d51	Speed cols missing, vehicle counts intact

**Pattern:** ALL 11 speed range columns are missing for the same rows. Vehicle type columns remain complete. This indicates speed sensor malfunctions while vehicle counting sensors continued working.

## 2. Data Validation

**Requirement from README:** Vehicle type counts should sum to speed range counts per row.

Year	Validation Result	Notes
2018-2023	✅ 100% match	Perfect data integrity
2024	⚠️ 99.92% match	Mismatches = rows with missing speed data
2025	⚠️ 99.11% match	Mismatches = rows with missing speed data
2026	⚠️ 99.25% match	Mismatches = rows with missing speed data

**Conclusion:** When speed data is missing, speed\_sum=0 but vehicle\_sum>0, causing validation to fail. This is expected given the sensor failure.

## 3. Yearly Volume Changes

Year	Rows	Change	Notes
2018	91,296	-	Baseline
2019	526,802	+477%	Massive network expansion
2020	1,294,200	+146%	Continued growth
2021	2,295,190	+77%	Peak growth
2022	2,252,484	-1.86%	Slight decline
2023	2,308,696	+2.50%	Recovery
2024	2,436,564	+5.54%	Peak volume
2025	2,189,654	-10.13%	Decline
2026	681,600	-68.87%	Partial year (Jan-Apr only)

## 4. ID Mapping File Issues

The provided LL [jaamad.xlsx](#) mapping file:

- Contains **119 stations with valid IDs**

- Has **27 rows with missing IDs** (periodic/non-permanent stations)
- **Missing IDs** that appear in traffic data:
  - 25d51 (2024-2026)
  - 0e481, ff2e7, fde61 (2025)

**Action Required:** Update mapping file or manually add these IDs.

## Converted Mapping Files

Created machine-readable CSV versions:

- `data/ll_jaamad_clean.csv` - 119 stations with valid IDs, clean column names
- `data/ll_jaamad_all.csv` - 146 stations including incomplete entries
- `data/ll_jaamad_id_mapping.csv` - Minimal mapping (id, name, lat, lon, road info)